

Inigo Quilez :: articles :: normal and areas of n-sided polygons - 2008



When doing looking for size optimization it's always a good idea to apply some basic algebra and expand expressions and try to factorize them again in a different way to see if the same result can be expressed in less code (usually at the expense of speed of accuracy). Calculating polygon normals and areas is one of those cases(say, for a mesh).

The triangle

Let's define a triangle ABC in 2 or 3D, defined by its three vertices **A**, **B** and **C**. We now want to compute its area or normal vector - remember that the area is half the length of its normal. The classic approach to computing it is to first compute the edge vectors **B-A** and **C-A** and then cross them:

$$\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C}) = (\mathbf{B} - \mathbf{A}) \times (\mathbf{C} - \mathbf{A})$$

and therefore

$$\text{area}(\mathbf{A}, \mathbf{B}, \mathbf{C}) = |\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C})|/2$$

where $|v|$ means the length of v . Now, let's expand the cross products, by distribution:

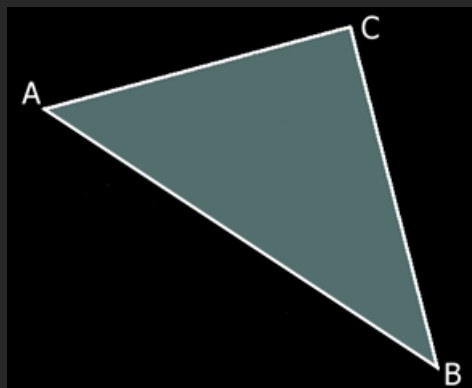
$$(\mathbf{B} - \mathbf{A}) \times (\mathbf{C} - \mathbf{A}) = \mathbf{B} \times \mathbf{C} - \mathbf{B} \times \mathbf{A} - \mathbf{A} \times \mathbf{C} + \mathbf{A} \times \mathbf{A}$$

Since the cross product is anticommutative, $\mathbf{B} \times \mathbf{A} = -\mathbf{A} \times \mathbf{B}$, and since $\mathbf{A} \times \mathbf{A} = 0$ (for it defines a zero area triangle), we get that

$$\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C}) = \mathbf{A} \times \mathbf{B} + \mathbf{B} \times \mathbf{C} + \mathbf{C} \times \mathbf{A}$$

which is the sum of one cross product per side of the triangle. We can

evaluate it by traversing all the vertices in order and cyclically, and doing the cross product with the following vertex. Not only this is interesting as an alternative way to compute triangle normals without edge vectors, but it also hits at some sort of generalization to polygons with more sides.



The quad

Let's define a quad ABCD by its four vertices **A**, **B**, **C** and **D**. We will compute both its surface area and normal as the sum of the areas and normals of two of the triangles it is composed of. This will work just fine for the surface areas, and for the normals too as long as the two triangles are coplanar and do define a real quad. In case they don't, the addition of the two normals will be nothing but the average normal of the quad. In any case, we have that:

$$\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}) = \text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{D}) + \text{normal}(\mathbf{B}, \mathbf{C}, \mathbf{D}) = \mathbf{A} \times \mathbf{B} + \mathbf{B} \times \mathbf{D} + \mathbf{D} \times \mathbf{A} + \mathbf{B} \times \mathbf{C} + \mathbf{C} \times \mathbf{D} + \mathbf{D} \times \mathbf{B}.$$

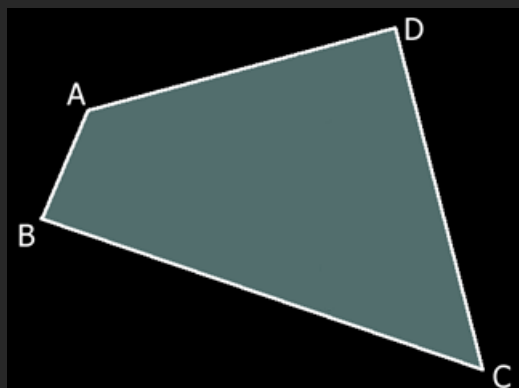
Again, due to the anticommutativity of the cross product ($\mathbf{B} \times \mathbf{D} = -\mathbf{D} \times \mathbf{B}$) we get that

$$\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}) = \mathbf{A} \times \mathbf{B} + \mathbf{B} \times \mathbf{C} + \mathbf{C} \times \mathbf{D} + \mathbf{D} \times \mathbf{A}$$

which is again a cyclic alphabetically sorted sum of one cross product per side of the polygon. Nice!

Of course, again, $\text{area}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}) = |\text{normal}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})|/2$

Interestingly, the normal/area of the quad can be also expressed as the cross of the diagonals: $(\mathbf{A}-\mathbf{C}) \times (\mathbf{B}-\mathbf{D})$. Indeed, if you apply distributivity and anticommutativity to this cross product, you will see it reduces to the same cyclic summatory of cross products.



And beyond

In fact, this is true for n-sided polygons. You can easily demonstrate this yourself by drawing a pentagon, then an hexagon, and generalizing the idea. The result is that for n vertices v_i with $i=\{0, 1, 2, \dots, n-1\}$ forming a polygon,

$$\text{normal}(v_0, v_1, \dots, v_{n-1}) = \sum v_i \times v_{i+1}$$

where of course the $i+1$ wraps to 0 when it otherwise becomes n .

Conclusions

This is extremely useful trick for 4 kilobyte demo coding, when you want to avoid doing some vertex subtractions in order to compute your face normals (and vertex normals too, as explained in the [article about clever 3D mesh normalization](#)). The problem this technique might have arises in the cases where the polygons or mesh are big and some of its vertices are far from the origin. In that case the cross products might be dealing with very parallel vectors and the precision of the result might be compromised. The regular method of

computing normals doesn't suffer from this as vertices get subtracted from each other. However, for most use cases, the method presented here works just fine. Promise.

inigo quilez - learning computer graphics since 1994